QUANTUM TRADING

12.16.2024

ALEX DAGHER

AGENDA

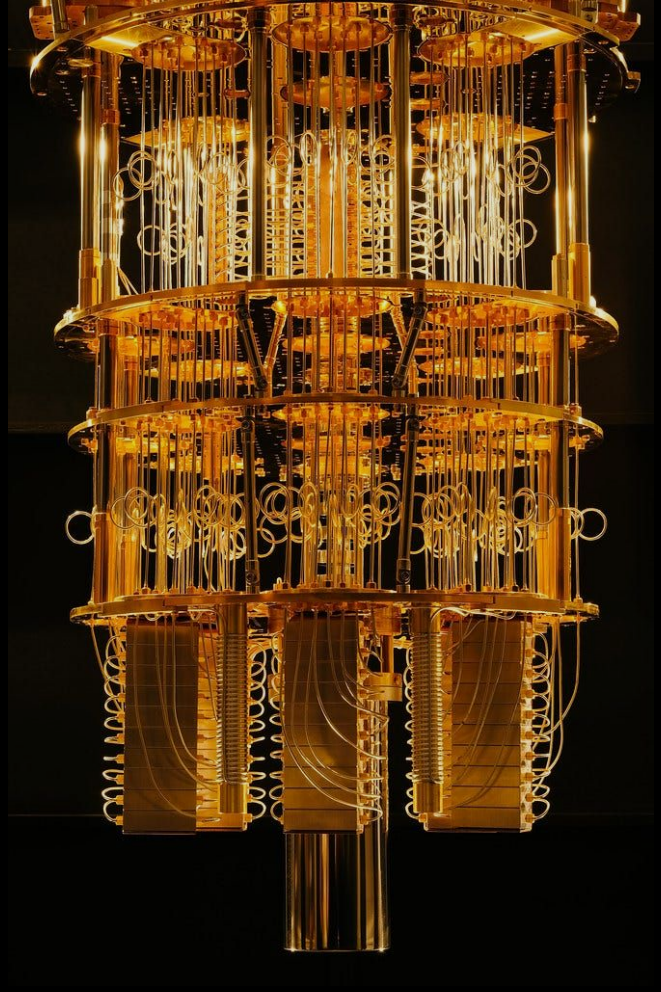
01 INTRODUCTION

02 PRODUCT OVERVIEW

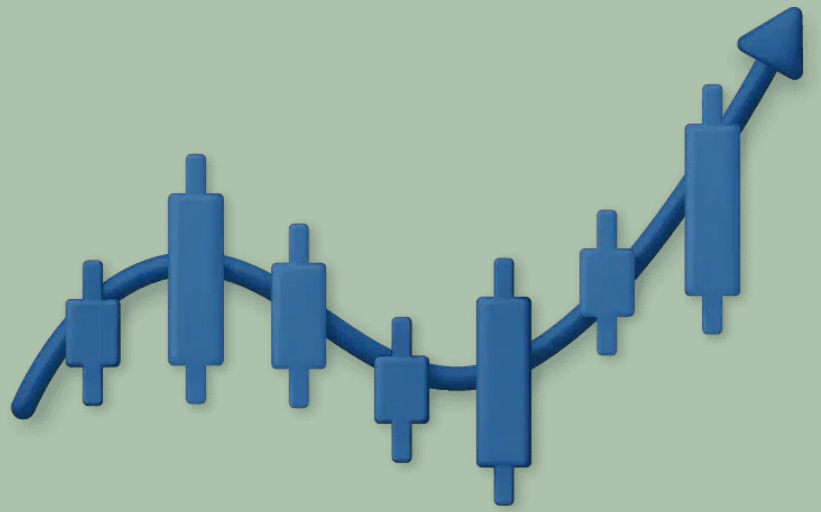
03 METHODOLOGY

04 RESULTS/ FINDINGS

05 FUTURE STEPS



1. APPLICATION OF QUANTUM COMPUTING ON FINANCE/TRADING
2. PROBLEM STATEMENT
3. PROBLEM SOLUTION



1 INTRODUCTION

Application of Quantum Computing on Finance/Trading

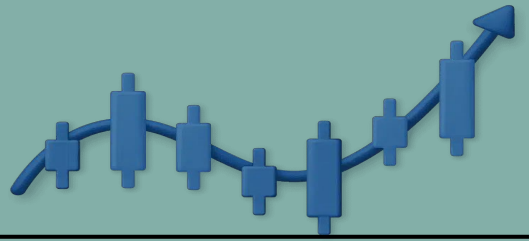
What is Quantum Finance?

- Quantum finance explores applying quantum computing principles to financial markets.
- Leverages quantum properties, like superposition and randomness, for unique computational approaches.

Why Markets?

- Financial markets are complex systems influenced by patterns and human behavior.
- Quantum computing's randomness aligns with the unpredictability of human-driven market dynamics.

Problem Statement



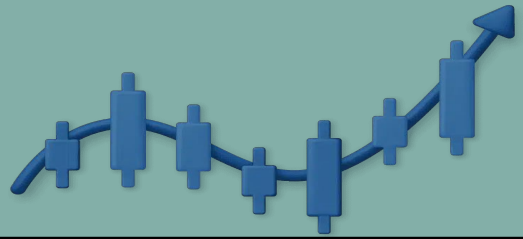
Classical ML Limitations

- Effective for simulations, trend analysis, and historical data modeling.
- Lacks the ability to incorporate human randomness in decision-making and trade execution.

Quantum Limitations

- Noisy and limited quantum hardware.
- Difficulty handling large data.
- Interpretability challenges.

Problem Solution



Classical ML

- LSTM: Captures temporal patterns in time-series data.
- XGBoost: Provides robust feature-based predictions.

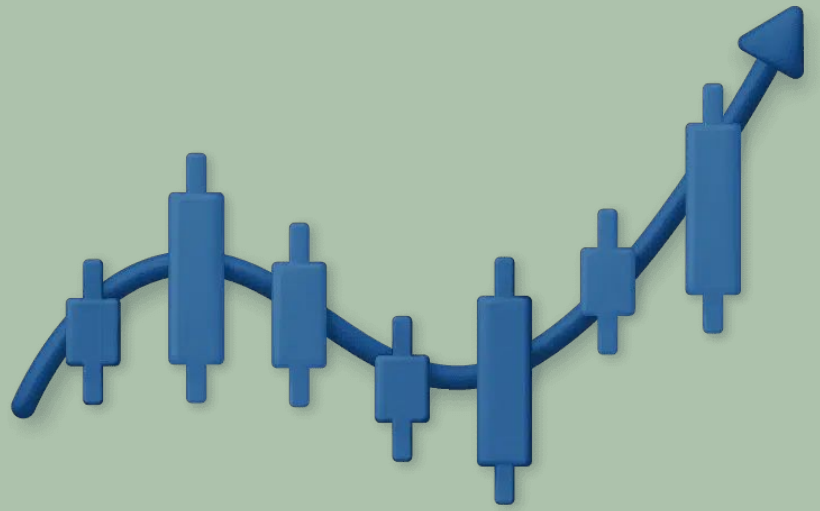
Quantum Encoding

- Angle Encoding: Encodes data into quantum states for high-dimensional representation.

Quantum Optimization

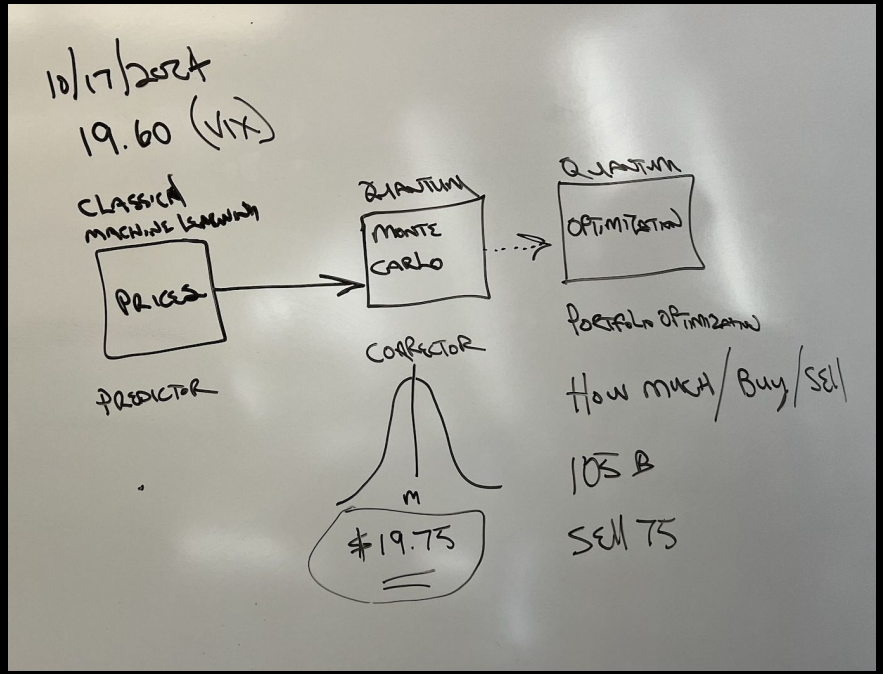
- Monte Carlo Optimization: Utilizes quantum randomness to explore potential outcomes.

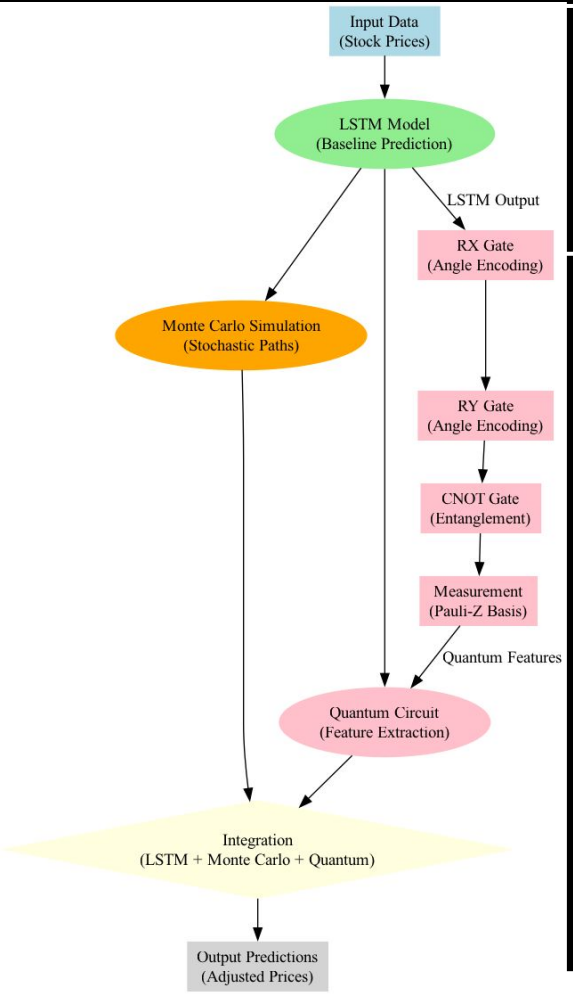
1. THE HYBRID MODEL
2. CODE WALKTHROUGH



2 PRODUCT OVERVIEW

THE HYBRID MODEL





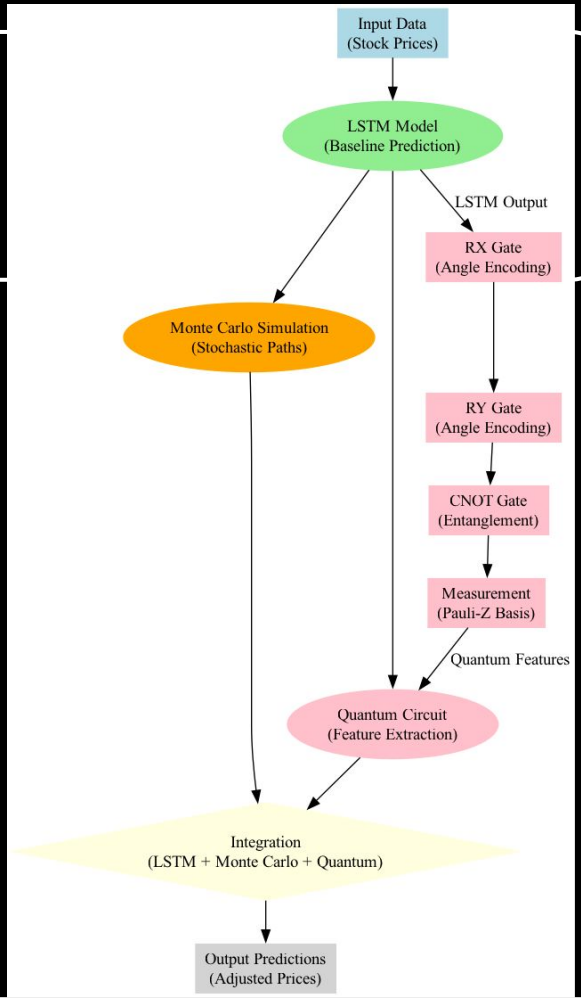
Code Walkthrough

INPUT DATA (STOCK PRICES)

```
# Step 5: Load and preprocess training data
dataset_train = pd.read_csv('NVDA - Training.csv')
training_set = dataset_train.iloc[:, 1:2].values

# Feature Scaling
from sklearn.preprocessing import MinMaxScaler
sc = MinMaxScaler(feature_range=(0, 1))
training_set_scaled = sc.fit_transform(training_set)

# Creating data structure with 60 timesteps and 1 output
X_train, y_train = [], []
for i in range(60, len(training_set_scaled)):
    X_train.append(training_set_scaled[i-60:i, 0])
    y_train.append(training_set_scaled[i, 0])
X_train, y_train = np.array(X_train), np.array(y_train)
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
```

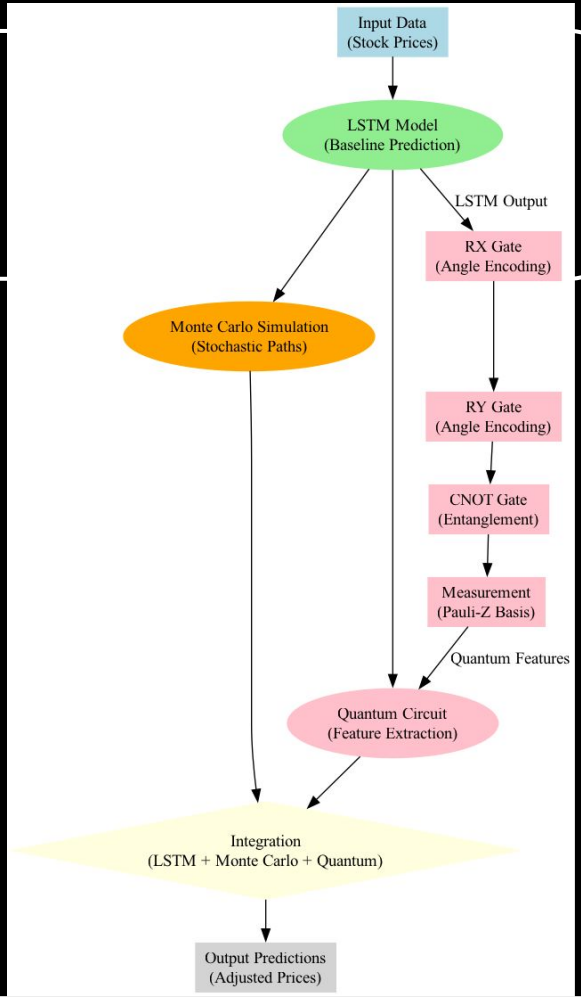


Code Walkthrough

LSTM MODEL (BASELINE PREDICTION)

```
# Step 6: Build the LSTM model
regressor = Sequential()
regressor.add(LSTM(units=50, return_sequences=True, input_shape=(X_train.shape[1], 1)))
regressor.add(Dropout(0.2))
regressor.add(LSTM(units=50, return_sequences=True))
regressor.add(Dropout(0.2))
regressor.add(LSTM(units=50, return_sequences=True))
regressor.add(Dropout(0.2))
regressor.add(LSTM(units=50))
regressor.add(Dropout(0.2))
regressor.add(Dense(units=1))

# Step 7: Compile and train the RNN
regressor.compile(optimizer='adam', loss='mean_squared_error')
regressor.fit(X_train, y_train, epochs=100, batch_size=32)
```

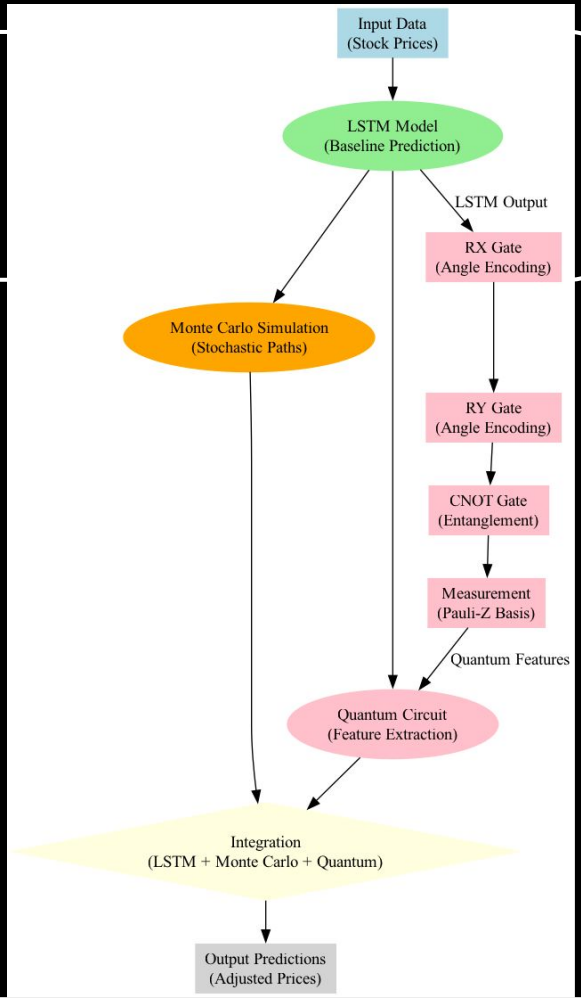


Code Walkthrough

QUANTUM CIRCUIT/ ENCODING

```python
# Step 4: Define the quantum circuit
@qml.qnode(dev)
def circuit(params):
    qml.RX(params[0], wires=0)  # Encode data using RX rotation
    qml.RY(params[1], wires=1)  # Encode data using RY rotation
    qml.CNOT(wires=[0, 1])      # Introduce entanglement
    return qml.sample(qml.PauliZ(0)), qml.sample(qml.PauliZ(1))

# Function to encode LSTM predictions into quantum circuit
def encode_data_to_quantum(predictions):
    encoded_results = []
    for pred in predictions:
        # Normalize prediction to fit into the range [0, 2*pi] for RX/RY gates
        normalized_pred = 2 * np.pi * (pred / np.max(predictions))
        params = [normalized_pred[0], normalized_pred[0]]  # Using same value for both qubits
        samples_0, samples_1 = circuit(params)
        encoded_results.append(np.mean(samples_0 + samples_1))  # Aggregate result as a feature
    return np.array(encoded_results)
```



Code Walkthrough

MONTE-CARLO SIMULATION

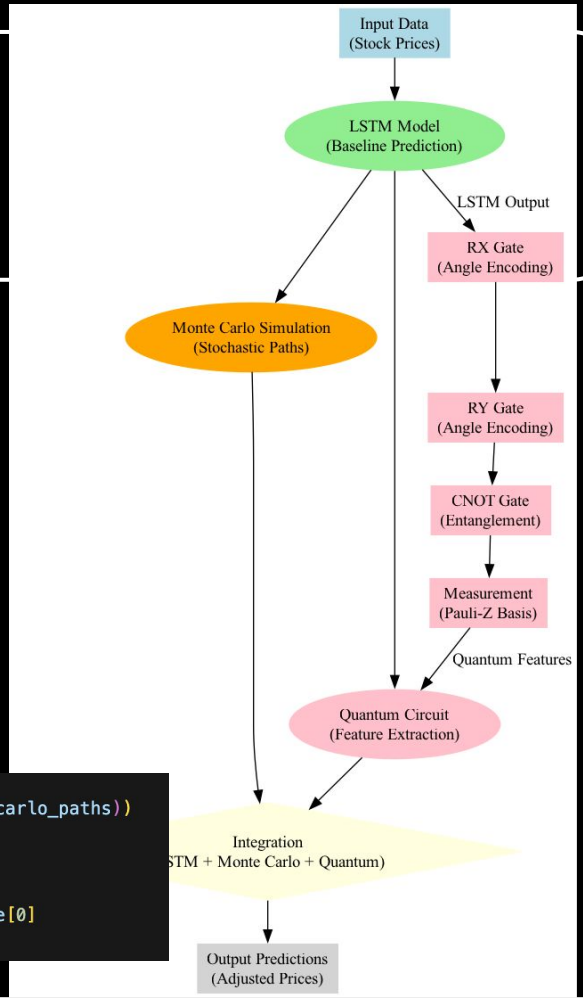
```
# Step 9: Perform Monte Carlo simulation for adjustments
adjusted_predictions = []
encoded_features = encode_data_to_quantum(predicted_stock_price)  # Encode LSTM predictions into quantum circuit
for idx, lstm_price in enumerate(predicted_stock_price):
    monte_carlo_paths = []
    for _ in range(1000):  # 100 Monte Carlo simulations per prediction
        prices = [lstm_price[0]]
        for _ in range(N):
            Z = np.random.normal(0, 1)
            S_t = prices[-1] * np.exp((mu - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z)
            prices.append(S_t)
        monte_carlo_paths.append(prices[-1])

    mean_monte_carlo = np.mean(monte_carlo_paths)
```

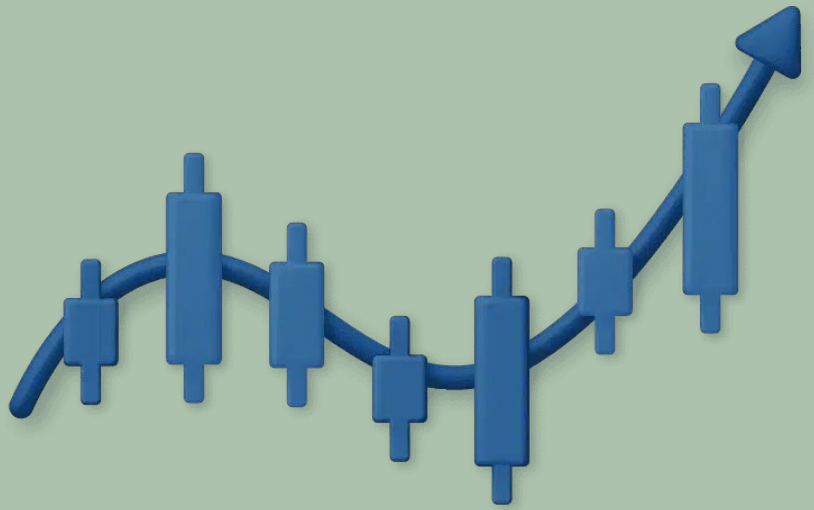
INTEGRATION

```
# Normalize Monte Carlo mean and quantum feature
monte_carlo_norm = (mean_monte_carlo - np.min(monte_carlo_paths)) / (np.max(monte_carlo_paths) - np.min(monte_carlo_paths))
quantum_norm = encoded_features[idx]  # Quantum output normalized between 0 and 1

# Combine LSTM, Monte Carlo, and Quantum adjustments
adjusted_price = 0.6 * lstm_price[0] + 0.2 * monte_carlo_norm * lstm_price[0] + 0.2 * quantum_norm * lstm_price[0]
adjusted_predictions.append(adjusted_price)
```

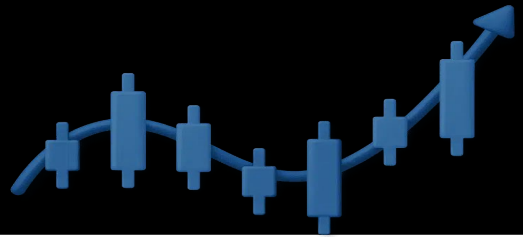


3



METHODOLOGY

METHODOLOGY



Data and Stock Selection:

- ➔ NVIDIA Stock (“NVDA”): Selected due to high volatility and relevance in tech-driven markets.
- ➔ Originally VIX

Model Development:

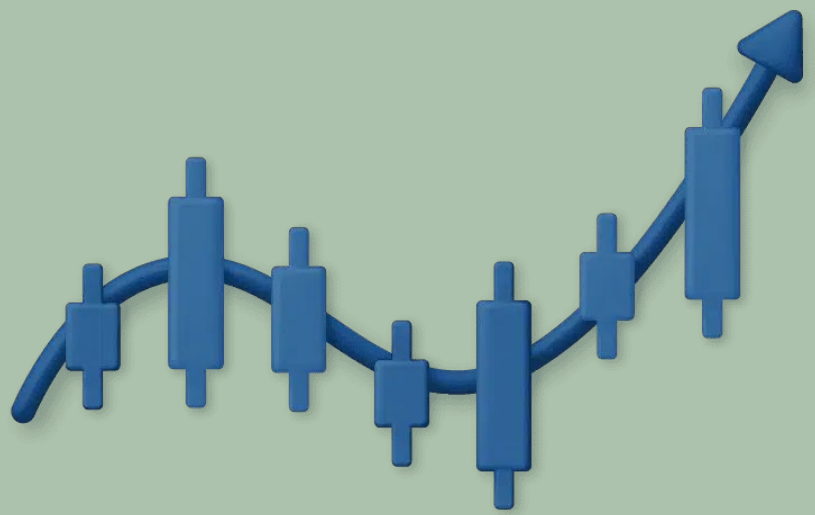
- ➔ Trained daily with 100 and 400 epochs
- ➔ Backtested using data from the 30 trading days preceding the current day.

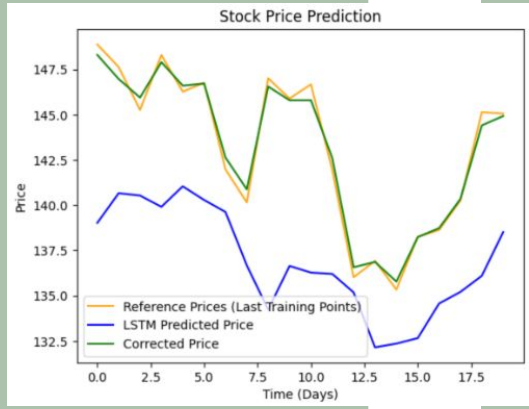
RECORDING/ ANALYSIS:

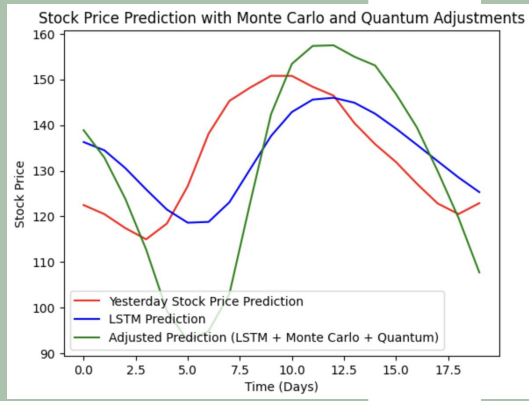
- ➔ Data Comparison: 100 vs 400 epoch Corrected vs LSTM
- ➔ PROFITABLE ?

1. OUTPUT
2. ANALYSIS

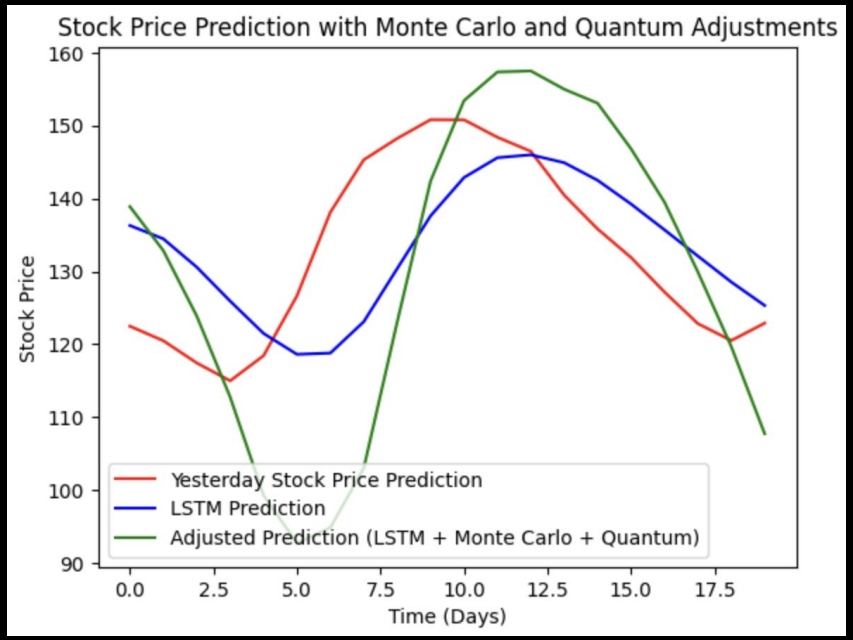
4 RESULTS AND FINDINGS

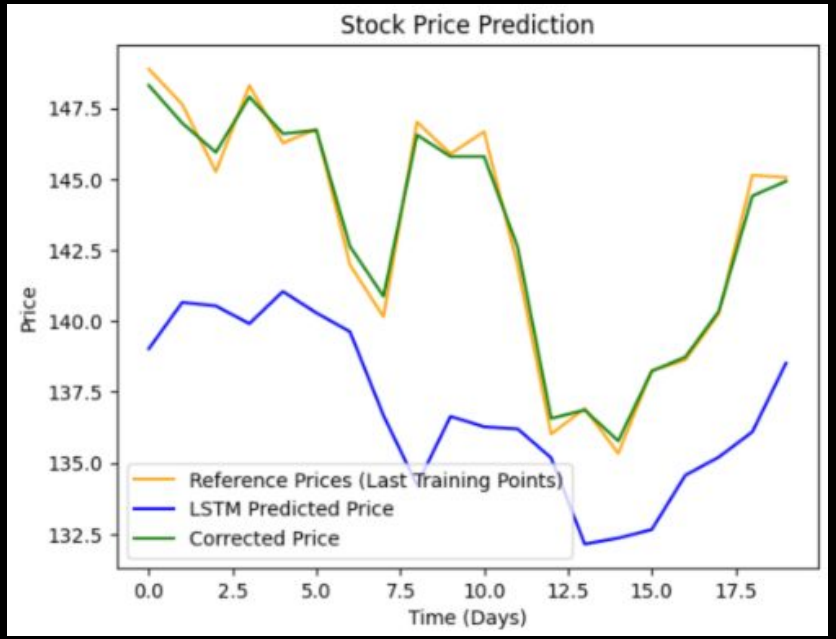






OUTPUTS


Stock Price Prediction with Monte Carlo and Quantum Adjustments
Stock Price
Time (Days)
Yesterday Stock Price Prediction
LSTM Prediction
Adjusted Prediction (LSTM + Monte Carlo + Quantum)


Stock Price Prediction
Price
Time (Days)
Reference Prices (Last Training Points)
LSTM Predicted Price
Corrected Price

100 vs 400 epoch

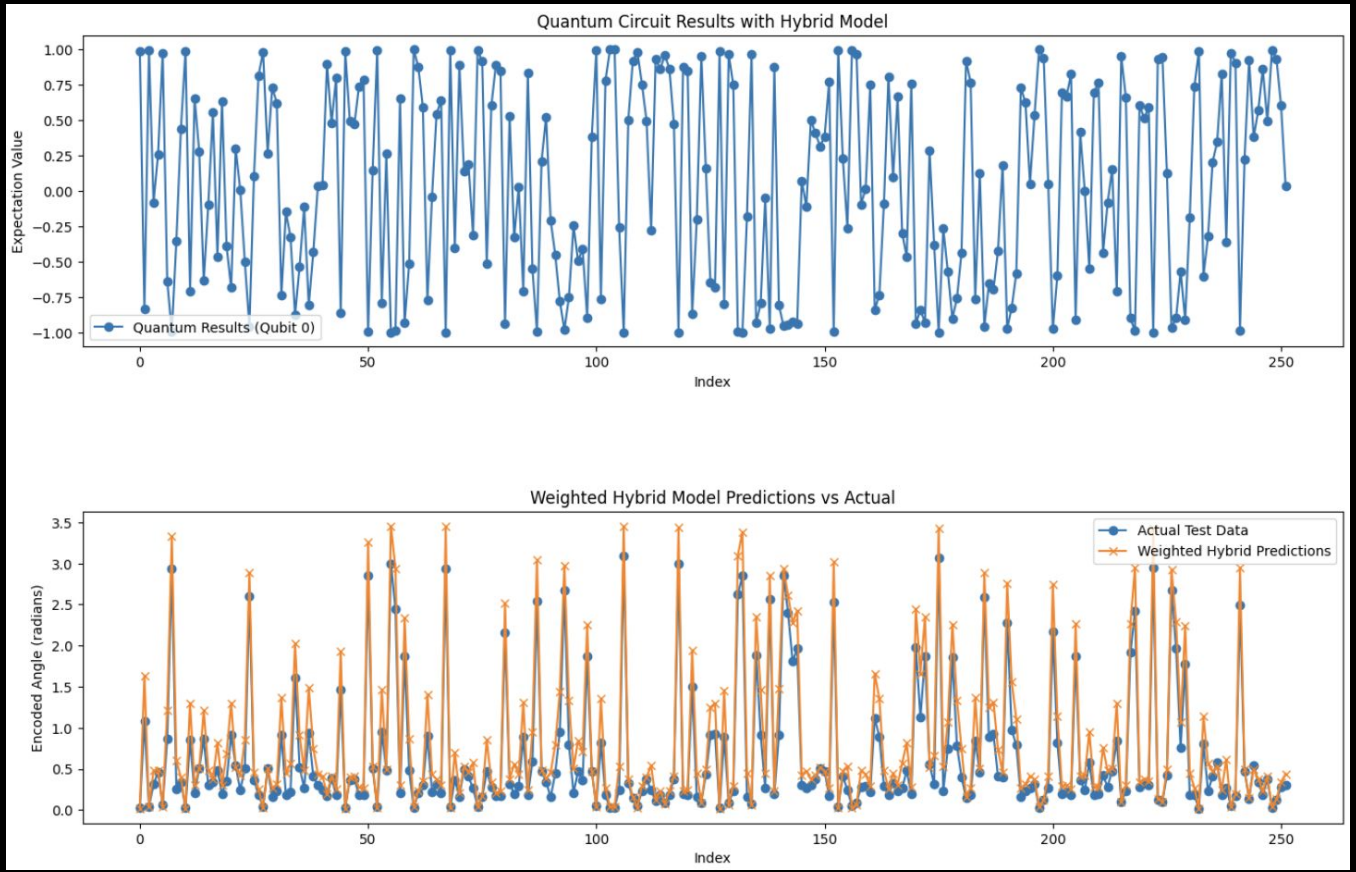
	A	B	C	D	E	F	G	H	I
1	Date	Open	Close	Predicted 100	Predicted 400	Winner	Predicted trend for the day	same direction	money maker?
2	Nov 15, 2024	144.87	141.98	142.6128845	144.5755005	100 epox		YES PUTS	Money
3	Nov 18, 2024	139.5	140.15	134.6609039	148.7776184	400 epox		NO	Breakeven
4	Nov 19, 2024	141.32	147.01	141.3063812	144.4035645	400 epox	Bullish	YES CALLS	Money
5	Nov 20, 2024	147.41	145.89	136.7800293	141.0664825	400 epox	Bearish	YES PUTS	Money
6	Nov 21, 2024	149.35	146.67	131.6764069	145.6890869	400 epox	Bearish	YES PUTS	Money
7	Nov 22, 2024	145.93	141.95	137.9915924	144.4847717	400 epox	Bearish	YES CALLS	Money
8	Nov 25, 2024	141.99	136.02	145.0969849	142.0498352	400 epox	Bullish	NO	Loss
9	Nov 26, 2024	137.7	136.92	141.2778625	144.8241882	0	Bullish	NO	Loss
10	Nov 27, 2024	135.01	135.34	142.2886047	140.6927948	400 epox	Bullish	YES CALLS	Breakeven
11	Nov 29, 2024	136.78	138.25	145.5605469	139.8892212	400 epox	Bullish	YES CALLS	Money
12									
13									
14									
15	Date	Open	Close	Predicted 100	Predicted 400	Winner	Predicted trend for the day	same direction	money maker?
16	Dec 2, 2024	138.83	138.63	135.5	139.5335846	/	Bullish	NO	Loss
17	Dec 3, 2024	138.26	140.26	129.1859131	136.9769135		Bullish	YES CALLS	Money
18	Dec 4, 2024	142	145.14	138.4177246	143.5609283		Bullish	YES CALLS	Money

LSTM VS ADJUSTED

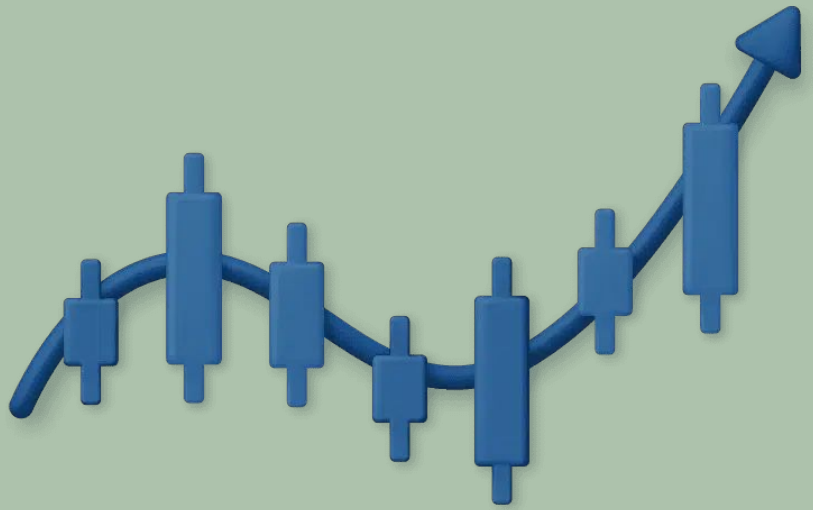
	A	B	C	D	E
1	Date	Open	Close	LSTM	MONTE_CARLO
2	Nov 27, 2024	135.01	135.34	137.2481995	152.4234433
3	Nov 29, 2024	136.78	138.25	142.7024231	138.3909539
4	Dec 2, 2024	138.83	138.63	134.1243286	134.4442259
5	Dec 3, 2024	138.26	140.26	141.3889008	131.1850686
6	Dec 4, 2024	142	145.14	139.7661285	122.4718704
7	Dec 5, 2024	145.12	145.06	139.6689148	138.8793639

CAN WE MAKE MONEY???

BONUS


Quantum Circuit Results with Hybrid Model
Expectation Value
Index
Quantum Results (Qubit 0)
Weighted Hybrid Model Predictions vs Actual
Encoded Angle (radians)
Index
Actual Test Data
Weighted Hybrid Predictions

5



FUTURE STEPS

1 Complete **hybrid model testing** on NVIDIA stock.
Execute trades based on hybrid predictions.
Compile a **portfolio** to demonstrate practical application

2 **Increase LSTM Training Data:**
Why? Deep learning models like LSTM require a substantial amount of training data for better generalization.

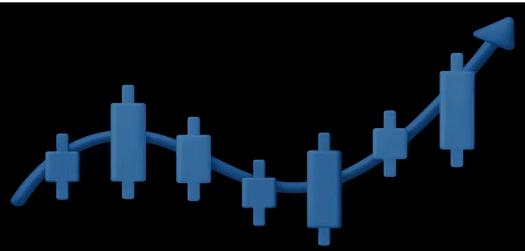
3 **Advanced Quantum Encoding,** Replace the current angle encoding with: **Amplitude Encoding**

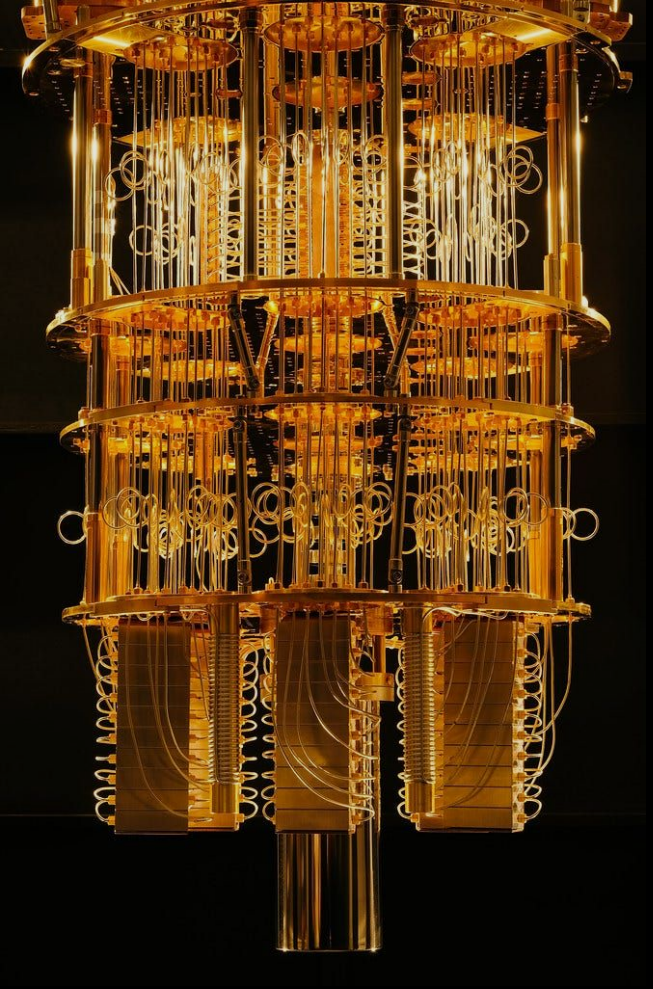
4 **Monte Carlo Simulations:**Increase the Number of Simulations, Use Alternative Stochastic Models:
Heston Model or **Jump Diffusion Models**

5 **Add Ensemble Learning:** Combine predictions from multiple models: **LSTM**, **XGBoost/Random Forest**, **ARIMA** (for traditional time series)
Average or weight their predictions intelligently to improve accuracy.

6 **Optimize Hyperparameters:**Number of LSTM units, Dropout Rate, Epochs and Batch Size

FUTURE PLAN





THANK YOU

QUANTUM TRADING